DriftConditions Implementation Plan

Spring 2026 UI/UX Upgrade

Purpose

This implementation plan outlines the next set of UI/UX improvements and critical fixes, focusing on streamlining the contributor workflow, improving listener display consistency, and introducing basic engagement features.

1. Goals and Scope

1.1. Goals

The primary goals of this upgrade are to:

1. **Improve Display and Usability:** Enhance consistency and clarity for listeners and editors across the Audio and Recipe views.
2. **Streamline Contributor Workflow:** Reduce friction and improve feedback loops for users submitting and managing audio content.
3. **Enhance Discovery and Navigation:** Introduce core functionality for searching and usage tracking within the audio library.
4. **Explore Listener Engagement:** Implement new features to gather immediate feedback on the listening experience.

1.2. Why these goals matter:

These improvements address several long-standing friction points in the Editorial Back End, making content management easier for editors/mods and improving the quality of life for contributors. Introducing basic discovery and engagement tools (like search and feedback) prepares the system for growth by improving usability as the audio library evolves.

1.3. Scope

This upgrade primarily focuses on frontend and API changes across the Audio List, Audio Edit, Recipe List, and Playlist components, broken down by category:

1.3.1. Polish & Display

- Change audio duration display from seconds to minutes:seconds in audio edit and audio view.

- Ensure the "View Recipe" page matches the width and style of the "Edit Recipe" page for consistency.
- Resolve the bug in the audio list where the play/stop icon remains visible after audio playback ends.
- Implement a success message confirmation after an audio file is uploaded.
- Ensure the correct image is displayed when audio is played on a phone device.
- Improve accuracy of time labels in the Playlist view by offsetting each displayed mix's timestamp by one position in the queue, compensating for the lag between Icecast queue time and actual playback start.
- Show number of times a clip has been used.

1.3.2. Contributor Experience

- Implement an email notification system to inform contributors when their submitted audio is approved.
- Add a pre-submission check during audio upload that warns the user if the file (based on filename and checksum) has already been submitted.
- Ensure that backend errors from the audio upload and audio edit routes are correctly passed back and displayed on the frontend form.
- Add a "quick edit" feature (similar to the one for roles) to the Recipe List page.

1.3.3. Discovery & Navigation

- Implement a full-text search capability for the audio list.
- Add tracking for how many times each audio clip is used in recipes and display this count on the audio view and edit pages.

1.3.4. Engagement (Stretch Goals)

- Implement a 'thumbs up/down' feature for the playlist component to gather immediate listener feedback.

1.4. Out of Scope:

- Any changes to the core streaming Mix Engine behavior.
- New recipe authoring functionality (beyond the quick-edit feature).
- Any changes to the completed Authentication and Authorization system.

2. Implementation: Polish & Display

2.1. Time Format Update

Audio duration is currently displayed as raw seconds in three places: the Audio List (formatted as a float with two decimal places), the Audio Edit page, and the Audio View page. The format is not user-friendly, so we will standardize it to minutes:seconds (e.g. `2:07`).

A shared `formatDuration(seconds)` utility function will be added to `AdminClient/src/utils/formatUtils.js`. It converts a numeric seconds value to a `m:ss` string, zero-padding the seconds. All three display sites — `AudioList.js`, `AudioEdit.js`, and `AudioView.js` — will be updated to call this function rather than formatting inline.

2.2. Recipe Page Consistency

The View pages for Audio and Recipe used a `view-wrapper` class that constrained the `display-box` to 640px, while the Edit pages used an `edit-wrapper` that set it to 850px. This created a jarring width shift when an editor navigated between viewing and editing the same record. Both `RecipeView.js` and `AudioView.js` were updated to use `edit-wrapper` so all four pages share the same 850px width. No CSS changes were required.

Additionally, the Recipe Edit and Recipe View pages were missing breadcrumb navigation items present on the Audio pages. Recipe Edit was updated to add "Add New" to its breadcrumb and to render the breadcrumb at both the top and bottom of the form, matching Audio Edit. Recipe View was similarly updated to add "Add New" and to render the breadcrumb at both top and bottom.

2.3. Audio Playback Icon Bug Fix

The play/stop icon in the Audio List is toggled by adding and removing an `active` CSS class directly on the button element. When playback ends naturally, the `onEnded` handler on the `<audio>` element updates the `playingAudio` state but does not remove the `active` class from the button, so the icon remains in the "playing" state.

The fix is to update the `onEnded` handler to also clear the `active` class from any `.listen button` elements, matching the cleanup already done at the top of `togglePlayAudio`.

2.4. Audio Upload Confirmation

Upon successful upload, the server returns the new audio ID and the client immediately navigates the user to the Edit page (for users with edit permissions) or the View page (for

others). This navigation serves as implicit confirmation that the upload succeeded — no additional success message is needed. This item is complete as-is.

2.5. Playlist Time Display

The playlist is returned ordered by `dateUsed DESC` (currently index 0). The display intentionally skips the current mix and shows the recent play history. To compensate for the lag between when a mix is queued to Icecast and when it actually begins playing, the time label for each displayed mix is offset by one position — showing the `dateUsed` of the following mix as an approximation of when that item ended. This adjustment was already in place and is sufficient for current purposes.

2.6. Public Profile Access

Profile pages are publicly accessible and linked from the playlist, but unauthenticated users were seeing "User Not Found." Two misconfigurations combined to cause this:

1. **Auth check rejected the page load** — `authRoutes.js /check` was missing `profile` from its `publicContexts` list, so unauthenticated users were turned away before the profile data could load.
2. **Profile route threw on missing token** — `userRoutes.js /profile` called `getRequestingUserInfo()` which throws when no token is present, landing in the catch block and returning an error instead of the profile.

Fix: Added `profile` to `publicContexts` in `authRoutes.js`. Updated the `/profile` route to gracefully handle a missing token — unauthenticated callers receive the `basic` field set (name, bio, location, role, member since) with no email, status, or edit controls.

3. Implementation: Contributor Experience

3.1. Duplicate Submission Check

To prevent redundant uploads, a duplicate check is performed server-side when an audio file is submitted — both through the single upload form and the bulk uploader.

Detection logic:

- Checksum match → reject. The exact file has already been submitted. No override.

Checksum: MD5, computed server-side from the file buffer before saving. Fast, dependency-free via Node's built-in `crypto` module, and sufficient for duplicate detection at any realistic library size. Collision probability is negligible (~1 in 3.4×10^{38}).

Schema change: A `checksum` column is added to the `audio` table. Existing records require a one-time backfill — iterating over existing files on disk and computing their checksums.

Single upload: A checksum rejection returns an error displayed on the upload form.

Bulk upload: The uploader iterates and reports per-file status. Checksum rejections are reported as **Duplicate** (red).

Graceful degradation: Until the backfill is complete, older files without checksums won't be caught by the duplicate check.

3.2. Backend Error Reporting

Error responses across the app were inconsistent — some routes returned plain text, others returned JSON objects of varying shapes, and most frontend components ignored the server message entirely and showed a hardcoded fallback.

The fix standardizes the entire chain to a single pattern:

Server — all error responses now return `{ error: { message } }` JSON. The only exception is the auth `/check` route, which already uses an extended `{ error: { code, reason, message } }` shape that the frontend relies on for navigation decisions, and is left unchanged. OAuth callback routes use redirects rather than JSON responses and are also unchanged.

Redux thunks — every catch block now extracts the error message the same way: `error.response?.data?.error?.message || 'Server error. Try again later.'` and passes that string to `rejectWithValue`. The rejection payload is always a plain string. The `checkPageAuth` thunk is the sole exception — it passes the full response so the auth flow can inspect the `reason` field.

Frontend pages — every `.catch()` handler now uses the payload string directly: `setError(err || 'Something went wrong.')`. The `ProfileEdit.js` string-matching hack for detecting "username taken" is removed — the server now sends a clear message that can be displayed as-is.

3.3. Recipe Quick Edit Feature

Add an inline quick-edit row to the Recipe List page, matching the pattern already established in Audio List. Clicking "Quick Edit" in a recipe's action list expands an edit row beneath that recipe without leaving the page.

Quick edit fields:

- **Title** — text input
- **Status** — dropdown (Review / Approved / Disapproved / Trashed)
- **Description** — textarea
- **Classification** — plain text input (comma-separated)
- **Tags** — plain text input (comma-separated)
- **Comments** — textarea

The full recipe JSON (`recipeData`) is intentionally excluded — structural recipe edits belong in the full Edit page.

Implementation: `RecipeList.js` gains the same quick-edit state and handler pattern as `AudioList.js`: `editRecipeID`, `editedRecord`, `updateTrigger`, `openEditRow`, and `handleSubmit`. Table rows are wrapped in `React.Fragment` to accommodate the conditional edit row. The existing `recipeUpdate` thunk in `recipeSlice.js` is reused with no server changes required.

4. Implementation: Discovery & Navigation

4.1. Full-Text Audio Search

To help contributors and editors find specific clips, a search input is added to the Audio List page. Search queries across title, tags, and comments, and stacks with existing filter, sort, and pagination controls — preserved in the URL like any other filter.

Search fields: title, tags, comments. Tags are stored as JSON strings in the database — a `LIKE` match on the raw column is sufficient. Classification is excluded (better handled by the existing filter buttons).

Search behavior: Multiple words are ANDed — all terms must match. Quoted phrases (e.g. `"water droplets"`) are treated as a single token matching the exact phrase. Each token is matched against title, tags, and comments.

UI: A search input and Submit button appear in the filter bar above the audio table. A Clear button appears when a search is active. Pressing Enter submits. Search term is preserved in the URL (`?search=term`) and survives sort and filter changes.

Backend: The `/api/audio/list` route accepts a `search` param. A small parser splits the query into tokens (respecting quoted phrases), then appends `AND (title LIKE ? OR tags LIKE ? OR comments LIKE ?)` per token.

No schema changes required.

5. Risk Management and Fallback Plan

This upgrade primarily involves frontend logic and API standardization, localizing most risks to user-facing display issues and potential regressions in core list components.

5.1. Primary Risks and Mitigations

Risk	Description	Mitigation / Fallback
UI Regressions	New features (Quick Edit, Search) break existing list page functionality (sorting, pagination, filtering).	Implement comprehensive unit and end-to-end testing focused on `AudioList.js` and `RecipeList.js` to prevent regressions.
Data Display Errors	Custom utility functions (e.g., `formatDuration`) display incorrect values (e.g., 2:07 displays as 127:00).	Manual QA and dedicated tests for all new data formatting utility functions to ensure M:SS and usage counts are correct across all contexts.
Backend Error Inconsistency	Some API failures return unparsable text errors instead of standard JSON, leading to a confusing "Something Went Wrong" message.	Enforce the standardized `{ error: { message } }` JSON structure for all server-side exceptions and use Redux thunks to reliably capture and display this message on the frontend.
Clip Usage Tracking Delay	Implementation of *4.2. Audio Clip Usage Tracking* is blocked by MixEngine dependencies (Appendix 8.3).	**Fallback:** Launch the UI/UX features without the usage count display. The implementation pages (Audio View/Edit) will gracefully show "Used 0 times" or suppress the field until the MixEngine dependency is resolved in a subsequent phase.

5.2. Fallback Procedure

If a critical production error is detected on the main `Audio List` or `Recipe List` pages, the fastest procedure is an immediate rollback to the previous stable release. Since most features are compartmentalized into distinct files, targeted feature flags (if implemented) or quick removal of the offending file/route are secondary options.

6. Verification and Success Criteria

Success for this project is defined by meeting the specific UI/UX and workflow improvements outlined in the scope. Verification requires observable proof that each component operates as intended for both editors and contributors.

6.1. Verification Criteria

Category	Success Criterion (Observed Result)
Polish & Display	Audio duration displays consistently in *M:SS* format across all Audio List, Edit, and View pages. The play/stop icon correctly clears its active state after natural playback ends.
Consistency	The View pages for Audio and Recipe match the wider 850px container of the Edit pages, eliminating visual "jump" when navigating.
Contributor Workflow	A contributor receives an automated email notification after their submission is approved. Submitting a known duplicate file triggers a pre-submission warning or rejection on the upload form.
Editing Efficiency	The Quick Edit row on the Recipe List page successfully saves changes to the title, status, description, and tags without a full page reload.
Discovery	Typing a term into the Audio List search box correctly filters the list by matching against the title, tags, and comments fields.
Usage Tracking	The total number of times an audio clip has been used is displayed accurately on both the Audio View and Audio Edit pages (dependent on MixEngine instrumentation).

7. Stretch Goals

7.1. Thumbs Up/Down

This feature will implement a "thumbs up/down" rating system within the playlist component to collect immediate listener feedback on the current audio. This explores basic listener engagement features to gauge the listening experience. It will also flag a playlist so it isn't deleted.

7.2. User List Quick Edit

Add an inline quick-edit row to the User List page, matching the pattern established in Audio List and Recipe List. Clicking "Quick Edit" in a user's action list expands an edit row beneath that user without leaving the page.

Quick edit fields:

- **Role** — dropdown (user / contributor / editor / mod / admin)
- **Status** — dropdown (Active / Inactive)

That's it. Name, email, username, and password changes have security and identity implications that belong in the full Profile Edit page. The quick-edit use case here is an admin or mod needing to promote/demote a role or toggle a status without a full page load — the two most common moderation actions.

Implementation: `UserList.js` gains the same quick-edit state and handler pattern (`editUserID`, `editedRecord`, `updateTrigger`, `openEditRow`, `handleSubmit`). Rows wrapped in `React.Fragment`. Reuses the existing `profileEdit` thunk in `userSlice.js` — or a new lightweight `userUpdate` thunk if `profileEdit` carries too much overhead. No server changes required beyond confirming the existing profile update route accepts partial records.

7.3. Contributor Profile Stats

The profile page now surfaces contribution statistics alongside the existing profile card, giving contributors a visible record of their impact on the station. This changes the contributor's relationship to DriftConditions — they can see their work accumulating over time, which encourages ongoing participation.

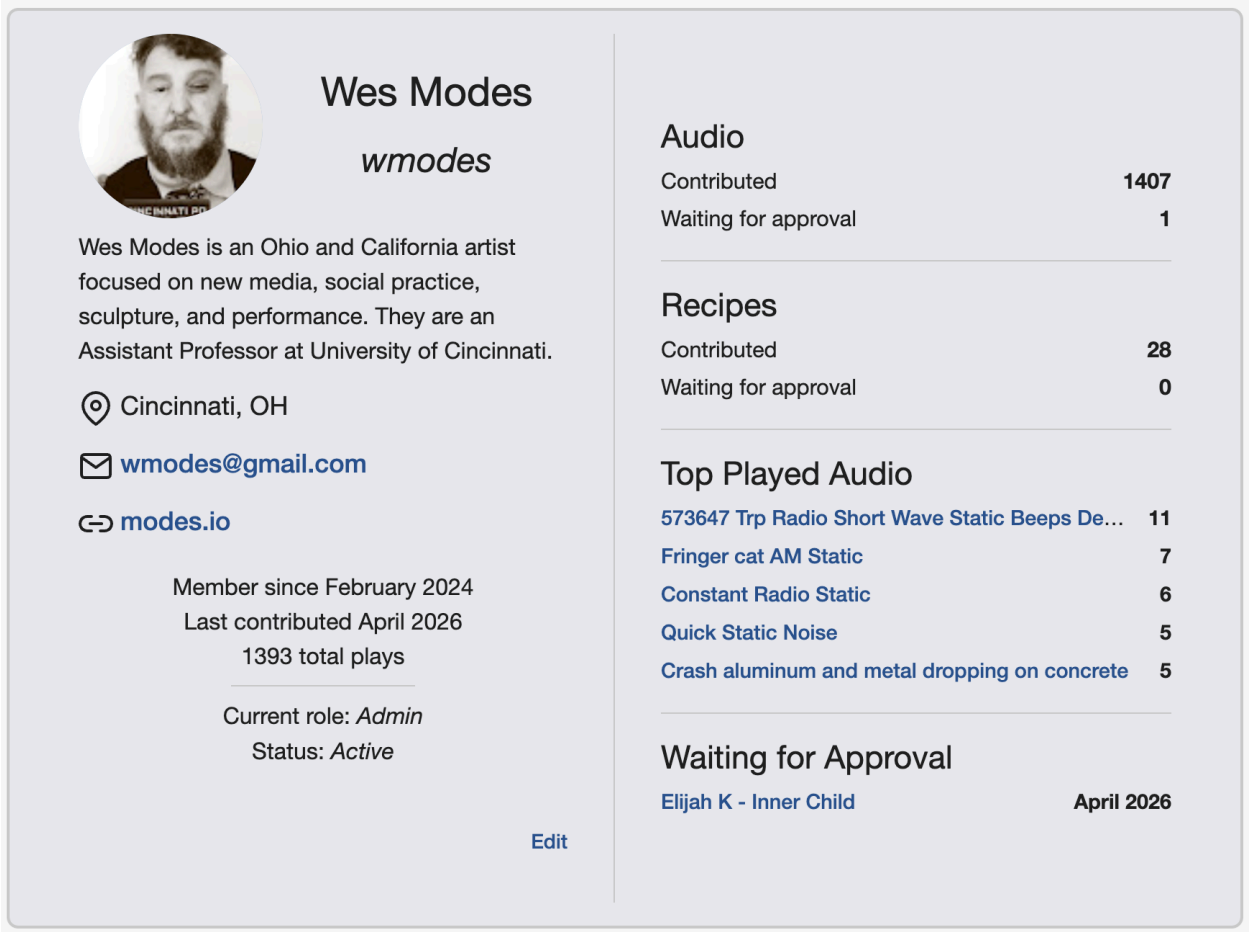
The profile card gains two additions in the footer: *Last contributed* date and *Total plays* across all their audio. The card is widened into a two-column layout, with a stats panel to the right separated by a vertical rule. The layout stacks into a single column on narrow screens.

Stats panel sections:

- **Audio** — contributed count; waiting for approval count (self/mod/admin only)
- **Recipes** (if any) — contributed count; waiting for approval count (self/mod/admin only)
- **Top Played Audio** — top 5 clips by play count; titles link to the audio view page for users with `audioView` permission
- **Waiting for Approval** (if any) — most recent 3 pending clips, visible only to the contributor themselves or moderators/admins

All stats are fetched in a single backend call added to the existing `/api/user/profile` route. A new `viewProfileExtras` permission gates the pending/approval data on other users' profiles; contributors automatically see their own pending data when viewing their own page.

Top audio and pending counts are configurable via `config.profile.topAudioCount` and `config.profile.recentPendingCount`.



8. Appendices

8.1. Appendix A — Database Schema Diff

8.1.1. Duplicate Submission Check (3.1)

To support duplicate detection at upload time, an MD5 checksum of each audio file is stored in the database. Existing records are backfilled via `scripts/backfill-checksums.js`.

```
ALTER TABLE audio ADD COLUMN checksum VARCHAR(32) NULL;
```

8.2. Appendix B — Server Changes

8.2.1. Duplicate Submission Check (3.1)

`AdminServer/routes/audioRoutes.js` — the `/upload` route now computes an MD5 checksum from the temp file buffer before moving it into place. It queries the `audio` table for a checksum match and rejects with 409 if found. It also checks for a filename match and returns `nameMatch: true` in the success response if found. The checksum is stored in the DB as part of the INSERT. Uses Node's built-in `crypto` module — no new dependencies.

8.3. Appendix C — Moved to Infrastructure / MixEngine Plan

The following tasks were scoped during this upgrade but deferred because they require MixEngine instrumentation and a new database table. They belong in the next MixEngine/ops improvement effort.

8.3.1. Audio Clip Usage Tracking

Recipe clips are selected dynamically at runtime by the MixEngine based on classification and length criteria — individual audio clips are not referenced by ID in recipe data. Accurate usage tracking therefore requires the MixEngine to instrument and log each clip it selects.

Implementation:

- New `clipUsage` table: `usageID` (PK), `audioID` (FK), `recipeID` (FK), `playedAt` (timestamp)
- MixEngine writes a row each time a clip is selected and queued for playback
- Backend adds a COUNT query against `clipUsage` to the `/api/audio/info` route, returning `timesUsed`

Considerations:

- Write performance under high playlist churn — `clipUsage` will grow rapidly; consider periodic aggregation or a counter cache column on the `audio` table
- Track at selection time or confirmed play time? Selection is simpler to instrument; confirmed play is more accurate but requires stream feedback from Icecast
- `clipUsage` also feeds usage-weighted clip selection (prefer less-used clips) and the contributor digest email (Appendix D)

8.3.2. Clip Usage Count Display

Once `clipUsage` exists, surface the count in the admin UI on Audio View and Audio Edit pages.

Implementation:

- `timesUsed` returned from `/api/audio/info` as a COUNT from `clipUsage`

- Audio View and Audio Edit display "Used X times" alongside duration and other metadata

8.4. Appendix D — Future Enhancements and Technical Debt

8.4.1. Mobile Playlist Album Art (Speculative)

On mobile, each new playlist could display a distinct visual identity — analogous to Spotify's per-album cover art — where the currently playing stream shows an image associated with that mix. This was originally noted as a potential display fix but is actually a net-new feature: it would require designing a per-playlist image asset system, storage, and a mobile-aware display component in the playlist UI. No current bug exists; this is a speculative UX enhancement for a future release.

8.4.2 Contributor Digest Email System

Rather than individual per-approval emails, a periodic digest re-engages contributors by summarizing their standing and recent activity on the station.

8.4.2.1. Motivation

Contributors tend to discover the station, interact, request contributor status, submit clips, and forget about it. Periodic communication maintains excitement and brings them back. A batch contributor who submits 10 clips in one session gets one digest, not 10 emails.

8.4.2.2. Digest Content

Personalized per user:

- Your username and link to your profile
- Your current role and what it means (contributor vs editor, etc.)
- Member since date
- *Since we last wrote:*
    - X clips approved and now in the mix (with titles + links)
    - X clips used in recipes (requires item 4.2 clip usage tracking)
- Possibly: total clips in the mix, station stats, a thank-you note

8.4.2.3. User Preference

Users can opt out of digest emails from their Profile Edit page. A boolean `digestEmail` field (default `true`) on the `users` table gates the digest runner — users with `digestEmail = false` are skipped entirely. Events are still queued in `userComms` in case the preference is re-enabled.

8.4.2.4. Cadence

Configurable — weekly default. Digest only sends if there is at least one pending event for that user.

8.4.2.5. Implementation

- `userComms` table + insert calls at event points (audio approved, clip used, role changed)
- Digest builder — groups pending comms by user, renders personalized email
- Scheduler — cron job or server interval firing the digest runner
- Uses existing `sendMail` from `utils/mailer.js`
- `sentAt` timestamp gates against re-sending; digest runner marks rows on send

8.4.2.6. userComms Table

An event queue accumulating communication-worthy events per user, decoupled from the digest sender:

column	type	notes
`commID`	int PK	
`userID`	int FK	recipient
`commType`	varchar	`audio_approved`, `clip_used`, `role_changed`, etc.
`payload`	JSON	contextual data (audioID, title, recipeID, etc.)
`createdAt`	timestamp	when the event occurred
`sentAt`	timestamp	null until included in a digest

8.4.2.7. Dependencies

Clip usage tracking (item 4.2) is needed to populate the "clips used in recipes" section. The digest can launch without it and add that row later.

8.5. Appendix E — Document Style Guide

This section details the standardized formatting applied to the document.

8.5.1. Headers and Section Titles

This style dictates the formatting for all section titles and headings (Heading 1, Heading 2, Heading 3, etc.) to ensure clear hierarchy and visual distinction from body text. All headers must be formatted exclusively using these standard Google Docs heading styles rather than custom text styles.

- **Font Family:** Arial
- **Font Size:** Varied (e.g., 20pt for Heading 1, 16pt for Heading 2, 14pt for Heading 3)
- **Font Weight:** Bold
- **Application:** Applied exclusively to section titles and subtitles, using the designated Docs Heading styles.

Structural Patterns: H1 headings (e.g., "1. Goals and Scope") must use the "Heading 1" doc style; H2 headings (e.g., "1.1. Goals") must use the "Heading 2" doc style; and this hierarchical pattern must be followed for all subsequent heading levels.

8.5.2. Body Paragraphs and Standard Prose

This section defines the baseline formatting for all narrative and descriptive text in the document. Standardized prose ensures optimal readability and consistency across all sections, distinguishing it clearly from headings and technical lists.

- **Font Family:** Arial
- **Font Size:** 11pt
- **Line Spacing:** 1.15
- **Spacing (Before Paragraph):** 0pt
- **Spacing (After Paragraph):** 12pt
- **Application:** Applied exclusively to normal body text, excluding headings, titles, and list items.

8.5.3. Inline Technical References

This style is used for all short code references embedded within the main text, such as variable names, file paths, API routes, and terminal commands. It serves to visually isolate code elements from surrounding narrative text.

- **Font Family:** Roboto Mono (Monospaced)
- **Text Color:** Gemini Green (#188038)
- **Background:** None
- **Use Cases:** Inline code snippets (e.g., `clipUsage` table), variable names (e.g., `audioID`), file paths (e.g., `config/config.js`), and console output text.

8.5.4. Code Block Syntax Highlighting Style: One Light (Notion variant)

A light-background syntax theme derived from the One Light family, originally developed for the Atom editor. Clean, low-contrast, and readable on white or near-white backgrounds.

Background: Light gray (`#f7f7f7` or similar off-white) **Default text:** Dark gray (identifiers, punctuation, table/column names)

Token colors:

- **Keywords** (`ALTER`, `CREATE`, `ADD`, `NOT`, `NULL`, `ON`, `DELETE`): Medium blue (`#0070c1` or similar)
- **Types** (`INT`, `VARCHAR`, `DATETIME`): Bright blue (`#0099cc` or similar)
- **Numeric literals** (`255`, `32`, `191`): Orange-red (`#e06c00` or similar)
- **Comments** (`-- ...`): Muted pink/mauve (`#c678a0` or similar)
- **Language label** (`SQL`): Small-caps, light gray, non-highlighted

Typography: Monospace, medium weight, generous line height. Token colors are distinct but not garish — optimized for readability over visual drama.

```sql
-- Existing 'users' table update (only adding utility columns)
ALTER TABLE users
  ADD COLUMN displayName VARCHAR(255),
  ADD COLUMN lastLoginAt DATETIME;

-- New 'userIdentities' table to support multiple linked OAuth accounts
CREATE TABLE userIdentities (
  userId INT NOT NULL, -- Foreign key to the users table
  oauthProvider VARCHAR(32) NOT NULL,
  oauthId VARCHAR(191) NOT NULL,
  PRIMARY KEY (oauthProvider, oauthId),
  FOREIGN KEY (userId) REFERENCES users(userID) ON DELETE CASCADE
);

-- Allow null names since OAuth providers may not supply them
ALTER TABLE users
```